

Eijen Hash Function

Keting Jia^{*} Qingliang Hou Guoxiao Liu Zhuoqing Peng
 Yantian Shen Xiaoen Lin
 Yukun Liu

^{*}Corresponding submitter: Keting Jia
Tsinghua University, Beijing, P.R.China
ktjia@tsinghua.edu.cn

Tsinghua University, Beijing, P.R.China
Zhongguancun Laboratory, Beijing, P.R.China
Shandong University, Qingdao, P.R.China
State Key Laboratory of Cryptography and Digital Economy Security, P.R.China

Contents

1	Introduction	4
2	Specification of Eijen	4
2.1	Notations	4
2.2	Recommended Instances and Parameter Sets	5
2.3	Sponge-F Mode	5
2.4	Specification of Permutation	7
2.4.1	MixQuad Layer	7
2.4.2	PosPerm Layer	9
2.4.3	AMix4 Layer	9
3	Security Claim	10
4	Security Analysis	11
4.1	Differential Cryptanalysis	11
4.2	Differential-Linear Cryptanalysis	11
4.3	Integral Cryptanalysis	12
4.4	Preimage Cryptanalysis	13
4.5	(Quantum) Collision Cryptanalysis	13
4.6	Other Cryptanalysis	14
5	Design Rationale	15
5.1	Sponge-F Structure	15
5.2	Design Choices for Permutation	16
6	Features	18
7	Implementation and Evaluation	18
7.1	Software Performance	19
7.2	Hash Performance	19
7.3	Summary of Performance Analysis	20
8	Randomness Tests	20
A	Round Constants	23

B	Test Vectors	24
B.1	Test Vectors for Eijen-256	24
B.2	Test Vectors for Eijen-384	27
B.3	Test Vectors for Eijen-512	28
B.4	Test Vectors for Eijen-1024	28

1 Introduction

In this document, we present the **Eijen** hash-function family. The design utilizes a feed-forward Sponge mode [1, 2] instantiated within a 2048-bit Add-Rotate-Xor (ARX) permutation, yielding a family of fixed-output hash algorithms **Eijen- h** , where h is the size of digest and $h \in \{256, 384, 512, 768, 1024\}$. The design has two goals. First, at the mode level, it aims to inherit the feature of feed-forward Sponge (Sponge-F) mode to break invertibility, and exact security claims against collision, preimage, and second-preimage attack in both classical and quantum settings. Second, at the permutation level, we use a 4×8 array of 64-bit words and word-based mixing operations to obtain fast diffusion and efficient software and hardware implementations.

2 Specification of Eijen

2.1 Notations

To avoid notational ambiguity, the following notations are adopted throughout this document:

- b denotes the state size in bits;
- r denotes the rate, representing the bit length of the message block;
- c denotes the capacity, which determines the security level of the algorithm;
- h denotes the bit length of the output hash digest;
- r' denotes the width squeezed from the capacity part;
- $\text{right}_i(X)$ denotes the rightmost i bits of X ;
- H_h denotes the fixed-output hash algorithm with a digest length of h bits;
- Π denotes the full 2048-bit permutation;
- $\mathcal{R}^{(i)}$ denotes the i -th round function of Π ;
- **MixQuad** denotes the column-wise word-mixing layer;
- **PosPerm** denotes the word position permutation layer;
- **AMix4** denotes the almost MixColumn layer.

2.2 Recommended Instances and Parameter Sets

This document specifies a family of hash algorithms **Eijen**, which is based on the feed-forward Sponge-F construction introduced in [1]. **Eijen** includes of five variants **Eijen- h** instantiated within a common 2048-bit ARX permutation, while different output digest size $h \in \{256, 384, 512, 768, 1024\}$. Table 1 summaries the five recommended instances and their corresponding parameter sets.

These parameter sets are carefully chosen to satisfy the $h \leq c$ constraint mandated for the fixed-output versions, while simultaneously maximizing the absorption rate. Specifically, all configurations adhere to a unified capacity formula $c = h + 64$, where $64 = \lceil \log_2 \ell_{\max} \rceil$ corresponds to the maximum number of message blocks $\ell_{\max} = 2^{64} - 1$. This design ensures that the second-preimage security, bounded by $\min\{h, c - \log_2 \ell\}$, maintains a full h -bit security level even at the maximum message length of $2^{64} - 1$.

Table 1: Recommended Instances of Eijen

Function	b	r	c	r'	h
Eijen-256	2048	1728	320	256	256
Eijen-384	2048	1600	448	384	384
Eijen-512	2048	1472	576	512	512
Eijen-768	2048	1216	832	768	768
Eijen-1024	2048	960	1088	1024	1024

2.3 Sponge-F Mode

Suppose the padded message is partitioned into ℓ blocks of size r bits:

$$\text{pad}_{10^*,r}(M) = M[0] \| M[1] \| \cdots \| M[\ell - 1], \quad M[i] \in \{0, 1\}^r.$$

Regarding message padding, we employ the injection Sponge-style padding rule

$$\text{pad}_{10^*,r}(M) = M \| 1 \| 0^k,$$

where $k \geq 0$ is the smallest integer such that

$$|M| + 1 + k \equiv 0 \pmod{r}$$

The message absorbing phase and the digest squeezing phase are illustrated in Figure 1. Let $X_i \in \{0, 1\}^b$ denote the internal state. The detailed absorbing and squeezing phases for **Eijen** hash function are shown as follows:

Initialization. The initial state is defined as

$$X_0 = 0^r \| IV,$$

where IV is defaulted to 0^c in **Eijen** hash function.

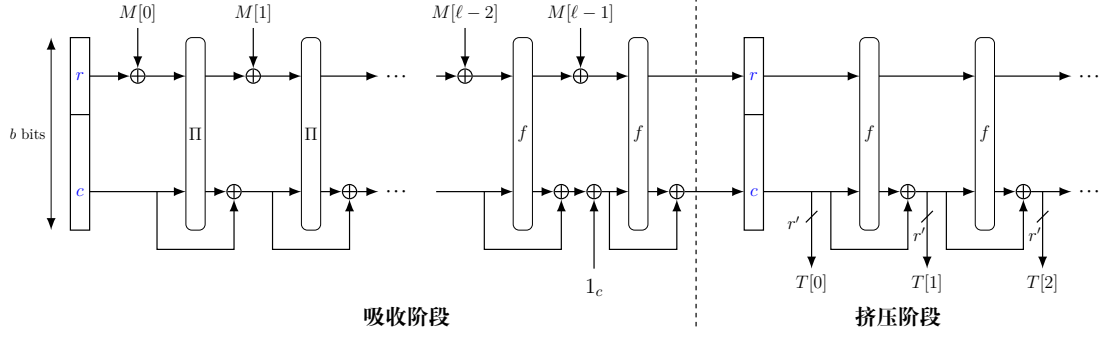


Figure 1: Sponge-F Mode

Absorbing Phase. For the message blocks before the last block, we define:

$$\begin{cases} S_{i-1} &= X_{i-1} \oplus (M[i-1] \parallel 0^c), \\ X_i &= \Pi(S_{i-1}) \oplus (0^r \parallel \text{right}_c(S_{i-1})), \end{cases}$$

for $1 \leq i \leq \ell - 1$. For the last message block, an additional constant 1_c is xored into the capacity part:

$$\begin{cases} S_{\ell-1} &= X_{\ell-1} \oplus (M[\ell-1] \parallel 1_c), \\ X_\ell &= \Pi(S_{\ell-1}) \oplus (0^r \parallel \text{right}_c(S_{\ell-1})) \end{cases}$$

where $1_c = 0^{c-1} \parallel 1$. Under little-endian byte ordering, the final 64-bit word of 1_c is represented as

$$1_c = (10000000, 00000000, \dots, 00000000).$$

Fixed-Output Extraction. For the fixed-output hash algorithm H_h , the digest is extracted from the capacity part of the final state:

$$H_h(M) = \text{right}_h(X_\ell). \quad (1)$$

Given that the rightmost c bits of the state correspond precisely to the capacity, this operation is equivalent to applying an h -bit truncation to the final capacity.

Full-Capacity Extraction. When $h = c$, the construction degenerates to outputting the entire c bits directly from the final capacity, yielding:

$$H_c(M) = \text{right}_c(X_\ell). \quad (2)$$

In the currently recommended parameter sets, all configurations satisfy the condition $c = h + 64 > h$. Consequently, the output digest invariably results from an h -bit truncation of the final capacity, rather than a full-capacity extraction.

Optional XOF Extension. If an extendable-output function (XOF) interface is required, one can repeatedly extract r' bits from the capacity part following the standard Sponge-F methodology. This extracted value is then fed forward into the rightmost r'

bit positions of the state for the subsequent round. However, as this document mainly focuses on fixed-output hashing, it omits a dedicated XOF specification for this concrete instantiation.

2.4 Specification of Permutation

The core component of Eijen is a permutation $\Pi : \{0, 1\}^{2048} \rightarrow \{0, 1\}^{2048}$ operating on a large-scale state of 2048 bits. The 2048-bit input string is rearranged into a $4 \times 8 \times 64$ three-dimensional array A . The state bit at index (i, j, k) within this array is denoted as $A[i][j][k]$, where $0 \leq i < 4$, $0 \leq j < 8$, and $0 \leq k < 64$. Let $A[i + 4 \cdot j] \in \mathbb{Z}_{2^{64}}$ represent the 64-bit word at index (i, j) . Furthermore, let A_j denote the four 64-bit words comprising the j -th column, defined as:

$$A_j = (A[4 \cdot j], A[4 \cdot j + 1], A[4 \cdot j + 2], A[4 \cdot j + 3])^T, \quad j = 0, 1, \dots, 7.$$

Each round function of the permutation consists of three distinct operations: the non-linear layer **MixQuad** (MQ), the word position permutation layer **PosPerm** (P), and the almost MixColumn layer **AMix4** (AM). The i -th round transformation $\mathcal{R}^{(i)}$ is defined as follows:

$$A^{(i)} \xrightarrow{\text{MixQuad}} Z^{(i)} \xrightarrow{\text{PosPerm}} U^{(i)} \xrightarrow{\text{AMix4}} W^{(i)} \xrightarrow{\text{PosPerm}} A^{(i+1)}$$

The permutation iterates for a total of 16 rounds, that is, $\Pi = \mathcal{R}^{(15)} \circ \mathcal{R}^{(14)} \circ \dots \circ \mathcal{R}^{(0)}$.

2.4.1 MixQuad Layer

The **MixQuad** operation is applied to each column of the internal state matrix, taking four 64-bit words (a, b, c, d) as input. For instance, when applied to the j -th column of the matrix, we have $(a, b, c, d) = (A[4 \cdot j], A[4 \cdot j + 1], A[4 \cdot j + 2], A[4 \cdot j + 3])$. Furthermore, we define the following left-rotation function on 64-bit words:

$$\text{ROTL}_{64}(x, n) = (x \ll n) \vee (x \gg (64 - n)).$$

Given the rotation parameters $\pi = (\pi_0, \pi_1, \pi_2, \pi_3, \pi_4, \pi_5)$ and the round constant **ci**, the **MixQuad** operation is illustrated in Figure 2 and is formally defined as follows:

$$\text{MQ}_{\pi, \text{ci}}(a, b, c, d) : \begin{cases} a \leftarrow a \boxplus b, & d \leftarrow d \oplus a, & d \leftarrow \text{ROTL}_{64}(d, \pi_0), & a \leftarrow \text{ROTL}_{64}(a, \pi_4) \oplus \text{ci}, \\ c \leftarrow c \boxplus d, & b \leftarrow b \oplus c, & b \leftarrow \text{ROTL}_{64}(b, \pi_1), & c \leftarrow \text{ROTL}_{64}(c, \pi_5), \\ a \leftarrow a \boxplus b, & d \leftarrow d \oplus a, & d \leftarrow \text{ROTL}_{64}(d, \pi_2), & \\ c \leftarrow c \boxplus d, & b \leftarrow b \oplus c, & b \leftarrow \text{ROTL}_{64}(b, \pi_3). & \end{cases}$$

Here, all additions $\boxplus$ are performed modulo 2^{64} . Compared to the ChaCha round function, the proposed design introduces an additional operation $a \leftarrow \text{ROTL}_{64}(a, \pi_4) \oplus \text{ci}$ in the first quarter-round, as well as an operation $c \leftarrow \text{ROTL}_{64}(c, \pi_5)$ in the second quarter-round. The design rationale for these modifications is as follows:

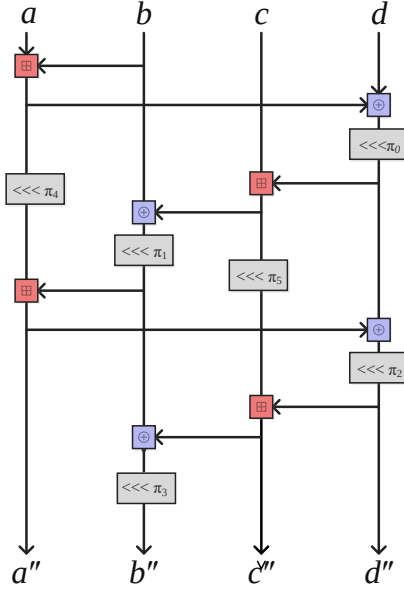


Figure 2: MixQuad Layer

- $\text{ROTL}_{64}(a, \pi_4) \oplus \text{ci}$: The additional rotation shifts the most significant bit (MSB) difference away from the MSB position, followed by the injection of a round constant. This modification breaks the deterministic differential propagation of words a and d at the highest bits during the inverse MixQuad operation (reducing the propagation probability from 1 in the original design to $\leq 2^{-1}$). Furthermore, the injection of the round constant effectively eliminates the vulnerability to slide attack.
- $\text{ROTL}_{64}(c, \pi_5)$: This rotation increases the diffusion depth of word c , thereby reducing the optimal probability of Δc in the forward differential (from 2^{-2} down to 2^{-3}).

For columns 0, 1, 2, and 3 (the left half) of the internal state matrix, the proposed design employs the following rotation parameters:

$$p_0 = (36, 54, 45, 44, 18, 9),$$

For columns 4, 5, 6, 7 (the right half) of the internal state matrix, the proposed design employs the following rotation parameters:

$$p_1 = (56, 46, 54, 34, 19, 51).$$

Therefore,

$$Z_j = \begin{cases} \text{MQ}_{p_0, \text{ci}_0^{(t)}}(A_j), & 0 \leq j < 4, \\ \text{MQ}_{p_1, \text{ci}_1^{(t)}}(A_j), & 4 \leq j < 8, \end{cases}$$

where $\text{ci}_0^{(t)}$ and $\text{ci}_1^{(t)}$ denote the round constants for the left and right halves in the t -th round, respectively.

Round Constants. Each round employs two 64-bit constants, which are injected into the **MixQuad** layer for the left and right halves, respectively. The entire set of 32 constants is derived from the hexadecimal expansion of the fractional part of π , extracted sequentially in 64-bit words:

$$\begin{aligned} \text{ci}_0^{(0)} &= \text{0x243F6A8885A308D3}, & \text{ci}_1^{(0)} &= \text{0x13198A2E03707344}, \\ \text{ci}_0^{(1)} &= \text{0xA4093822299F31D0}, & \text{ci}_1^{(1)} &= \text{0x082EFA98EC4E6C89}, \\ & \dots & & \\ \text{ci}_0^{(15)} &= \text{0xB4CC5C341141E8CE}, & \text{ci}_1^{(15)} &= \text{0xA15486AF7C72E993}. \end{aligned}$$

The complete list is provided in the Appendix A. All constants are publicly verifiable, ensuring full transparency and auditability of their generation process. Any interested party can independently reproduce this procedure to verify its correctness.

2.4.2 PosPerm Layer

Let Z denote the input to the word position permutation layer **PosPerm**, and let U be its corresponding output. The specific permutation mapping is defined as follows:

$$U[l] = Z[\sigma(l)], \quad l = 0, 1, \dots, 31,$$

where

$$\begin{aligned} \sigma = [22, 0, 29, 19, 26, 4, 1, 23, 30, 8, 5, 27, 2, 12, 9, 31, \\ 6, 16, 13, 3, 10, 20, 17, 7, 14, 24, 21, 11, 18, 28, 25, 15]. \end{aligned}$$

Equivalently, this can be expressed in matrix form as:

$$Y = \begin{bmatrix} 22 & 26 & 30 & 2 & 6 & 10 & 14 & 18 \\ 0 & 4 & 8 & 12 & 16 & 20 & 24 & 28 \\ 29 & 1 & 5 & 9 & 13 & 17 & 21 & 25 \\ 19 & 23 & 27 & 31 & 3 & 7 & 11 & 15 \end{bmatrix}.$$

2.4.3 AMix4 Layer

Let U be the input to the almost MixColumn layer **AMix4**. This layer performs a left-multiplication of each input column by a mixing matrix to produce the output W . Let the j -th column of U be represented as:

$$U_j = (U[4 \cdot j], U[4 \cdot j + 1], U[4 \cdot j + 2], U[4 \cdot j + 3])^T.$$

Define the column-mixing matrix as

$$L = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}.$$

The output column W_j is computed as follows:

$$W_j = L \cdot U_j,$$

where the multiplication is performed over the word-level field $\mathbb{F}_2$. Consequently, the operation is implemented via XOR as:

$$\begin{cases} W[4j] &= U[4j+1] \oplus U[4j+2] \oplus U[4j+3], \\ W[4j+1] &= U[4j] \oplus U[4j+2] \oplus U[4j+3], \\ W[4j+2] &= U[4j] \oplus U[4j+1] \oplus U[4j+3], \\ W[4j+3] &= U[4j] \oplus U[4j+1] \oplus U[4j+2]. \end{cases}$$

3 Security Claim

In this design, the newly defined 2048-bit permutation Π is combined with the Sponge-F paradigm to construct a hash algorithm. The adopted Sponge-F mode, with a digest length of h , features provable security against collision, preimage, and second-preimage attacks. Specifically, it provides approximately $h/2$ bits of collision resistance and approximately h bits of preimage resistance. For a challenge message with ℓ blocks after padding, it offers approximately $\min\{h, c - \log_2 \ell\}$ bits of second-preimage resistance.

Security Statement for the Current Instantiation. For the Eijen instantiation specified in this document, we claim that the design achieves the aforementioned security goals. Specifically, the 16-round permutation Π provides sufficient resistance against specialized structural attacks, differential cryptanalysis, linear cryptanalysis, meet-in-the-middle attacks, and other cryptanalytic methods. Accordingly, the security strength claims for Eijen are summarized in Table 2.

Table 2: Security Strength of Eijen

Function	h	Collision Security	Preimage Security	Second Preimage Security
Eijen-256	256	128	256	256
Eijen-384	384	192	384	384
Eijen-512	512	256	512	512
Eijen-768	768	384	768	768
Eijen-1024	1024	512	1024	1024

Scope of the Claims. This document primarily focuses on the security goals of the fixed-output hash algorithm family regarding collision, preimage, and second-preimage resistance. In addition, the assessment covers partial-collision attacks, semi-free-start collision attacks, free-start collision attacks, multi-collision attacks, length-extension attacks,

and attacks targeting any fixed subset of the output bits. It does not provide a specific indistinguishability proof for the current instantiation, nor does it derive security conclusions for the extendable-output function (XOF) in this section.

4 Security Analysis

4.1 Differential Cryptanalysis

Differential cryptanalysis is one of the most powerful cryptanalysis techniques for symmetric-key ciphers. The first step for the evaluation of the security against differential attack is to find differential trails with probability much higher than for a random function. Since the operation of differences is linear, the differences propagation through linear operations is non-probabilistic and deterministic. While the propagation of differences for non-linear operations are probabilistic.

In the underlying permutation Π of Eijen, the only nonlinear layer employs an improved quarter-round (QR) function of ChaCha, which is based on modular additions. In the extensive literature, various automatic methods has been proposed to model the probabilistic differences propagation through modular additions [3, 4]. However, the efficiency of automatic model is highly limited by the number of variables and constraints. In this paper, we use the SAT-based automatic tool proposed in [4] to evaluate the security of permutation Π against differential attack. Since the block size of Π is vary large, i.e., 2048-bit, the size of model is too large to be solved in a feasible time when attacked round $r > 2$. Instead of searching for differential characteristics with best differential probability (DP), we bound on the number of active words to significantly simplify the searching space. Based on our searching strategy, a 5-round differential characteristic were at most found with probability of 2^{-1903} . The final results are shown in Table 3. Despite many bounds about the number of active words are tried, we could not find a valid 6-round differential characteristic.

Table 3: Differential distinguishers for round-reduced Π

Attacked rounds	1	2	3	4	5
Number of active words	8	22	38	60	98
$-\log_2$ DP	8	34	194	1263	1903

4.2 Differential-Linear Cryptanalysis

This subsection gives differential-linear distinguishers against round-reduced version of the permutation Π . To this end, a r -round permutation E is divided into three parts:

$E = E_2 \circ E_m \circ E_1$, where a r_1 -round differential trail $\Delta_{in} \rightarrow \Delta_m$ is used in E_1 , a r_m -round differential-linear approximation $\Delta_m \rightarrow \Gamma_m$ is used in E_m , and a r_2 -round linear trail $\Gamma_m \rightarrow \Gamma_{out}$ is used in E_2 , such that $r = r_1 + r_m + r_2$. Let p denote the differential probability of the differential trail of E_1 , r denote the correlation of the differential-linear approximation of E_m , and q denote the correlation of the linear trail of E_2 . For the permutation E , a r -round differential-linear approximation $\Delta_{in} \rightarrow \Gamma_{out}$ with correlation $\text{cor} = prq^2$ can be obtained. Assume that the input and output spaces of E are both $\mathbb{F}_2^n$. If $\text{cor} \gg 2^{-n/2}$ is satisfied, one can use the above r -round differential-linear approximation to distinguish E from a pseudorandom permutation.

The search for differential-linear distinguishers adopts a typical hourglass structure. Concretely, for E_m we traverse all combinations of 1-bit input differences $\Delta_m = [i]$ with 1-bit or consecutive 2-bit output linear masks $\Gamma_m \in \{[j], [j, j+1]\}$; For each combination we extend Δ_m (resp. Γ_m) backward (resp. forward) and automatically search for the optimal differential trail (resp. linear trail) in E_1 (resp. E_2). Setting $(r_1, r_m, r_2) = (1, 2, 2)$, the best 5-round differential-linear distinguisher we obtained has a correlation $2^{-32.83}$. For the configuration $(r_1, r_m, r_2) = (2, 2, 2)$, since the differential probability weight is large, the time cost of the search procedure is high. In order to construct a 6-round distinguisher, we extend the above 5-round distinguisher backward by 1 round in the differential part. Finally, we got a 6-round differential-linear distinguisher with correlation $2^{-176.83}$. Details of the 2 distinguishers are given in Table 4. It can be seen in Table 4 that when the round number of the differential trail in E_1 increases from 1 round to 2 rounds, the differential probability decreases from 2^{-8} to 2^{-152} . This shows the fast diffusion of Π 's round function.

Table 4: Differential-linear distinguishers for round-reduced Π

Round number r	(r_1, r_m, r_2)	(p, r, q)	(Δ_m, Γ_m)	Correlation
5	(1, 2, 2)	$(2^{-8}, 2^{-0.83}, 2^{-12})$	$([1343], [1975])$	$2^{-32.83}$
6	(2, 2, 2)	$(2^{-152}, 2^{-0.83}, 2^{-12})$	$([1343], [1975])$	$2^{-176.83}$

4.3 Integral Cryptanalysis

The concept of integral cryptanalysis was first formalized by Knudsen and Wagner [5]. The core idea of integral cryptanalysis is finding a multiset of inputs such that the sum of some fixed parts of the resulting outputs is constant. The multiset of inputs in bit-based integral analysis usually consists of inputs taking all possible combinations in ℓ input bits, whereas the remaining $n - \ell$ bits take a fixed value. In the context of permutation, the output can be represented a vectorial Boolean function of input variables. If the degree of Algebraic Normal Form (ANF) of some output bits is less than ℓ , then these output bits are sum to zero, which can distinguish a cryptographic permutation from a random permutation. Division Property (DP) is a generalized integral cryptanalysis which was first introduced

by Todo [6] at Eurocrypt 2015, which was subsequently utilized to break the full MISTY1 cipher at CRYPTO 2015 [7]. Later, Xiang et al. pioneered the application of automated tools for searching bit-based division property [8]. Building upon this foundation, Sun et al. extended these automated searching methods to the bit-level cryptanalysis of ARX ciphers [9].

The hash function **Eijen** introduced in this paper use an ARX construction as the non-linear component in the underlying permutation. Hence, we use the SAT-based automatic searching tools proposed by Sun et al. [9] to evaluate the division property. After setting the i -th bit indexed by $\{0, 1, \dots, 2047\} \setminus \{64\}$ as active bits, and fixing the 64-th bit as constant, we found a 4-round distinguished trail where the sum for each of the output bits is zero. And the experiments reveal that it is practically impossible to construct a 5-round integral distinguisher.

4.4 Preimage Cryptanalysis

For Sponge-F construction, a more appropriate perspective is to view the final absorption step as a truncated Davies–Meyer (DM) hashing mode. Hence, we can use the enhanced Meet-in-the-Middle (MitM) techniques to evaluate the security of **Eijen** against preimage attack.

For the permutation Π of **Eijen**, any non-zero difference in a single input word could achieves full diffusion after 3 rounds. Specifically, the active columns in each round can be represented as $1 \rightarrow 3 \rightarrow 8$, where the total 8 columns after the **MixQuad** layer in the third round are fully diffused. Partial matching [10] utilizes any determined relations between two neutral sets in the states at the matching point. To ensure at least one-bit valid matching point, the number of rounds used for the partial matching is upper bounded by $5((3 - 1) + (3 - 1) + 1)$. Initial structure (IS) [11] is a few consecutive starting steps, where the two independent chunks overlapped. To ensure at least one neutral bit outside of the IS, the number of rounds used for IS is also upper bounded by 3. The Splice-and-Cut [10] attack framework is very suitable for DM hashing mode, and allows an attacker to add more 3 rounds in the worst case. In summary, at most 11-round ($3 + 3 + 5$) MitM preimage attack may be feasible, leaving a security margin of $16 - 11 = 5$ rounds.

4.5 (Quantum) Collision Cryptanalysis

For hash functions, the critical issue is not merely whether high-probability differential paths exist within the underlying permutation, but rather whether these paths can be translated into collision-generating differentials within the full Sponge-F mode. As discussed in Section 4.1, at most a 5-round differential characteristic for permutation with probability of 2^{-1903} can be found, thus the full-round **Eijen** is secure enough against (quantum) collision attack and (Semi-) Free-Start collision attack.

4.6 Other Cryptanalysis

This subsection evaluates attack methods explicitly considered in the submission evaluation criteria. The analysis is method-oriented: for each attack class, we identify the attack target induced by the Sponge-F mode and then relate the expected complexity to the exposed digest length h , the capacity c , the number of padded message blocks ℓ , and, when applicable, the fixed output subset size t .

Joux-Style Multi-Collision Attacks. The classical Joux multi-collision method applies to Merkle–Damgård iterations by repeatedly finding two-block alternatives that collide to the same n -bit chaining value. Repeating this process for s stages yields a 2^s multi-collision with an expected cost of about $s \cdot 2^{n/2}$, rather than the much larger direct birthday cost for all 2^s messages at once.

Joux-style multi-collision attacks are applicable to the Sponge-F iteration at the capacity-chaining level. This is a concrete consequence of the Sponge-F parameter choice: in order to increase the absorption rate, the capacity c is intentionally shorter than the capacity that would typically be required by a standard Sponge construction targeting the same preimage and second-preimage levels. Therefore, Joux-style multi-collisions can be constructed on the c -bit capacity chaining value; the correct question is whether the resulting $c/2$ -bit work factor remains above the claimed security target. For Eijen, the relevant chaining width is c , not h . More specifically, if S_{i-1} denotes the state after message injection in an absorbing step, then the capacity part after the feed-forward update is

$$B_i = \text{right}_c(\Pi(S_{i-1})) \oplus \text{right}_c(S_{i-1}).$$

Thus, a Joux-style step must find two message-block choices leading to the same c -bit capacity chaining value,

$$B_i = B'_i,$$

not merely to the same final h -bit digest. The expected complexity is about $s \cdot 2^{c/2}$ for a 2^s multicollision. Thus, the multi-collision resistance of the Sponge-F iteration is governed directly by the selected capacity length. Since all recommended instances use $c = h + 64$, this cost is

$$s \cdot 2^{c/2} = s \cdot 2^{h/2+32},$$

which remains 32 bits above the claimed $h/2$ -bit collision-security target. Table 5 lists the single-stage capacity-collision cost for the recommended instances; a 2^s multi-collision multiplies the corresponding entry by approximately s . The table should therefore be read as an explicit evaluation of the constructible Joux attack, rather than as a claim that Sponge-F rules out multi-collisions.

Length-Extension Attacks. In a Merkle–Damgård hash function, a length-extension attack uses the public digest as an exposed chaining value and continues the compression

Table 5: Joux-style capacity-collision cost for Eijen

Function	h	c	Single-Stage Cost
Eijen-256	256	320	2^{160}
Eijen-384	384	448	2^{224}
Eijen-512	512	576	2^{288}
Eijen-768	768	832	2^{416}
Eijen-1024	1024	1088	2^{544}

process on attacker-chosen suffix blocks. This method relies on the digest containing enough internal state information to resume hashing.

The Eijen digest is not an exposed chaining value. It is an h -bit truncation of the final capacity part, while the full 2048-bit state and the rate part remain unknown. Moreover, the recommended parameter sets reserve $c - h = 64$ additional capacity bits that are not included in the digest, and the final absorbing block injects $1_c = 0^{c-1}||1$ into the capacity part. This final-block constant separates the last absorption step from ordinary message-block absorption. Consequently, an attacker who knows only $\mathbf{H}_h(M)$ cannot deterministically reconstruct a valid internal state from which to continue absorbing an extension.

5 Design Rationale

The Eijen design targets high-security hashing under next-generation commercial cryptography requirements. The design considers both security requirements and software and hardware implementation constraints, and it separates the mode-level structure from the underlying permutation. This layered presentation keeps the hash mode and permutation design relatively independent, which simplifies specification, implementation, and cryptanalysis.

5.1 Sponge-F Structure

Feed-forward Breaks Invertibility. A fundamental characteristic of the standard Sponge construction is that the underlying permutation Π is public and invertible. Therefore, given any b -bit intermediate state, an attacker can evaluate it simultaneously in both forward and backward directions, thereby allowing direct stitching between consecutive permutation calls. This invertibility serves as the core point of exploitation for preimage and second-preimage attacks on the standard Sponge. Classical attacks utilize Π^{-1} to trace backwards during the squeezing phase, and then perform a meet-in-the-middle match with the forward search in the absorbing phase [12, ?].

Enhanced Security Bounds and Efficiency. Compared to the standard Sponge construction, the collision and (second) preimage security of Sponge-F can be reduced to simple and well-defined correlation intractability properties of the underlying permutation, which enables the establishment of precise and meaningful security bounds in both classical and quantum settings. Besides, compared to Sponge-DM [2], Sponge-F could reach identical security levels using a significantly smaller permutation state, thereby effectively compensating for the feed-forward overhead and making it highly suitable for constrained devices.

Large-State Digest Squeezing. The design requirements of China’s next-generation commercial hash functions specify 1024-bit digests and 1024-bit preimage and second preimage security. And standard sponge construction cannot achieve such a high level of security. While Sponge-F can provide an instance with $h = 1024$ and $c = 1088$, yielding 1024-bit preimage and 1024-bit second preimage security.

5.2 Design Choices for Permutation

With the goal of approximating a random permutation, we design a concrete permutation instance. We opt to instantiate it using a large 2048-bit state permutation. Such a substantial state size allows for higher absorption rates without significantly compressing the capacity. All configurations uniformly adhere to the capacity formula:

$$c = h + 64$$

where $64 = \lceil \log_2 \ell_{\max} \rceil$ ensures that the second-preimage security still achieves the full h bits even under the maximum message length of 2^{64} blocks. For instance, in the 256-, 384-, 512-, and 768-bit configurations, the capacities are 320, 448, 576, and 832 bits, respectively, with corresponding rates of 1728, 1600, 1472, and 1216 bits. All of these maintain a large absorption width. For the 1024-bit configuration, where

$$(r, c) = (960, 1088)$$

although the rate is somewhat reduced, it remains sufficient to provide reasonable throughput. The permutation function employs an iterated round-function structure.

The design of the round function thoroughly accounts for strong diffusion properties and compatibility with CPU parallel instructions. The round function is defined as follows:

$$\mathcal{R} = \text{PosPerm} \circ \text{AMix4} \circ \text{PosPerm} \circ \text{MixQuad}.$$

The isolated MixQuad layer operates locally on a column-by-column basis; while convenient to implement, it is insufficient to provide global diffusion. The first positional permutation breaks this column locality, the column mixing layer amplifies the activity within each affected column, and the second positional permutation then redistributes the amplified activity across the full width of the state. This decomposition strategy aims to achieve rapid qualitative diffusion while preserving implementation regularity.

The design of **MixQuad** improves upon the QuarterRound (QR) of ChaCha by avoiding backward paths with a probability of 1. Furthermore, path probabilities and bit diffusion properties were comprehensively considered during parameter selection. Regarding the overall structural design, we chose to utilize two distinct sets of **MixQuad** parameters. This prevents the uniform repetition of eight completely isomorphic columns, thereby mitigating the probability of identical local structures emerging in the left and right halves of the state. Concurrently, in combination with the subsequent structural components, this design avoids differential cancellation and enhances overall diffusion. The selection of the two sets of rotation parameters adheres to the following design criteria:

1. **Value Constraints:** The six values π_0 – π_5 are mutually distinct and all fall within the range $[3, 62]$.
2. **Distance Constraints:** The cyclic distance between π_4 and π_0 is ≥ 3 ; the cyclic distance between π_5 and π_1 is ≥ 3 .
3. **Modular Addition Constraints:** $\pi_1 + \pi_2 \not\equiv 0$, $\pi_2 + \pi_3 \not\equiv 0$, $\pi_3 + \pi_5 \not\equiv 0$, and $\pi_0 + \pi_j \not\equiv 0$ ($j = 1, 2, 3, 5$), all $(\text{mod } 64)$.
4. **Algebraic Constraints:** $(\pi_1 + \pi_2) \text{ mod } 64 \in [20, 40]$, $(\pi_3 + \pi_2) \text{ mod } 64 \in [20, 40]$, $\pi_1 + \pi_2 \not\equiv \pi_5$, $\pi_0 + \pi_1 \not\equiv \pi_4$, $\pi_3 + \pi_5 \not\equiv \pi_1$, $\pi_3 + \pi_2 \not\equiv \pi_1$, and $\pi_0 + \pi_1 \not\equiv \pi_4 + \pi_3 + \pi_2$, all $(\text{mod } 64)$.
5. **Single-Bit Diffusion Optimization:** Through forward and inverse experimental evaluations of 256 types of single-word, single-bit differences, the parameter sets with the highest comprehensive scores are selected from the candidates satisfying the aforementioned constraints.

The parameter construction of the **PosPerm** permutation is designed to break column locality while accounting for software instruction support and diffusion properties. It ensures that the four words of a single column are scattered into distinct columns. Subsequently, through the **AMix4** operation, one active column influences 12 words, and a second positional permutation further diffuses this influence across all columns. That is,

$$\sigma = [22, 0, 29, 19, 26, 4, 1, 23, 30, 8, 5, 27, 2, 12, 9, 31, \\ 6, 16, 13, 3, 10, 20, 17, 7, 14, 24, 21, 11, 18, 28, 25, 15].$$

Let ρ denote the function computing the output position for an input word. Then,

$$\rho(4j + i) = 4((j + s_i) \text{ mod } 8) + t_i,$$

where

$$(s_0, s_1, s_2, s_3) = (0, 1, 3, 4), \quad (t_0, t_1, t_2, t_3) = (1, 2, 0, 3).$$

Consequently, the four words within a single input column are mapped to four distinct output columns:

$$\{j, j + 1, j + 3, j + 4\} \pmod{8}.$$

6 Features

The design requirements for hash algorithms within China’s next-generation commercial cryptography framework are as follows:

1. The algorithm shall support at least two hash output lengths, 512 bits and 1024 bits, with the option to support a 768-bit output length. The maximum supported message length shall be at least $2^{64} - 1$ bits.
2. The algorithm shall be secure against known classical and quantum attacks.
3. The algorithm shall be efficiently implementable across a broad range of software and hardware platforms, ensuring high practical applicability.

The design of the proposed algorithm adheres to the following principles:

1. Security: The algorithm shall provide preimage resistance, second-preimage resistance, and collision resistance, thereby ensuring resistance against known cryptanalytic methods. The design also consider common attacks such as meet-in-the-middle (MitM) attacks and algebraic attacks.
2. Implementation efficiency: The hash algorithm can be executed efficiently across a wide range of software and hardware platforms. On software platforms, it is suitable for optimized 32-bit and 64-bit implementations; on hardware platforms, it take advantage of modular addition to achieve a highly favorable trade-off between compact circuit area and execution speed.
3. Flexibility: The algorithm is designed to adapt to different application requirements and operational environments. Subject to the stated security requirements, it supports multiple output digest lengths, such as 256, 384, 512, 768 and 1024 bits, for applications requiring different security levels.
4. Algorithmic simplicity: The implementation remains simple enough for precise specification, review, and deployment. Reducing implementation complexity lowers the risk of implementation errors and supports deployment in resource-constrained environments.

7 Implementation and Evaluation

This section presents the software implementations of Eijen, alongside an empirical performance comparison with SHA-3 (based on Keccak-f[1600]) in both standalone hashing and the SPHINCS⁺ signature scheme. All software benchmarking was conducted on an Intel Xeon Silver 4416+ processor, compiled using GCC with the `-O3 -march=native -flto` optimization flags enabled.

7.1 Software Performance

Scalar Reference Implementation. The state is represented as an array of 32 64-bit words. The **MixQuad** layer is executed column-by-column in-place, with each column updated through a small number of modular additions, XORs, and rotations. The **PosPerm** layer is executed in-place via the cycle decomposition of σ (four cycles of length 6 plus four swaps), thereby avoiding the need for temporary arrays. The **AMix4** layer is implemented using explicit XOR formulas for its four outputs.

AVX2 / AVX-512 Vectorized Implementations. Leveraging SIMD instruction sets, each row of the 4×8 matrix is loaded into a vector register (AVX2: four **ymm** registers; AVX-512: four **zmm** registers). The 8 columns of the **MixQuad** layer can be processed simultaneously across the 4 registers: modular additions use `_mm256/512_add_epi64`, and XORs use `_mm256/512_xor_si256/512`. AVX-512 utilizes the native `_mm512_rol_epi64` instruction (a single-instruction rotation), whereas AVX2 requires simulation using 3 instructions (shift + shift + OR). Round constants are injected using constant vectors constructed per round.

PosPerm exploits the row-preserving property of the σ table: all 8 elements of each output row originate from a single input row. Each **PosPerm** operation requires only 3 vector shuffles and 1 register copy.

For short messages common in SPHINCS⁺ (single-rate blocks), a SIMD-native fast path is implemented: the vector state is constructed directly from message bytes, bypassing scalar conversion and sponge buffering overheads. This fast path supports 4-way parallelism in AVX2 and 8-way parallelism in AVX-512.

7.2 Hash Performance

Table 6 presents a comparison of the single-hash latency (in cycles) and parallel throughput (cycles/hash, amortized). The test message length is 512 bits, and the digest length is 256 bits.

Table 6: Standalone hash performance (512-bit message, 256-bit digest)

Implementation	Parallelism	Cycles	Speedup vs. SHAKE-256 ref
SHAKE-256 ref	1	3,338	1.00×
Eijen ref	1	922	3.62×
SHAKE-256 AVX2	1	998	3.34×
SHAKE-256 AVX2	×4	535	6.24×
Eijen AVX2	×4	258	12.94×
Eijen AVX-512	×8	210	15.90×

The Eijen scalar reference implementation is already 3.6 times faster than the SHAKE-

256 scalar baseline, while the AVX-512 implementation achieves approximately a 16-fold speedup under 8-way parallelism. This advantage primarily stems from the large 2048-bit state of **Eijen**, which enables a 512-bit message to be completely absorbed within a single permutation call (rate $r = 1728$ bits $\gg 512$ bits), whereas the 24-round Keccak-f[1600] permutation is inherently slower.

Table 7 provides the single-hash latency across various message lengths.

Table 7: Single-hash latency across different message lengths (cycles, 256-bit digest)

Message (bits)	SHAKE-256 ref	Eijen ref	SHAKE-256 AVX2	Eijen AVX2
512	3,338	922	998	2,582
1024	1,902	912	1,000	1,150
2048	2,834	1,778	1,944	2,234
4096	5,180	2,628	3,818	3,368
8192	10,368	4,342	7,690	5,626

The **Eijen** scalar reference implementation outperforms the SHAKE-256 scalar baseline across all tested message lengths. When the message length exceeds the rate capacity (> 216 bytes, i.e., > 1728 bits), multiple absorption blocks are required. In such scenarios, the significant rate advantage of **Eijen** allows it to maintain a 2.4-fold performance edge even for 8192-bit messages.

7.3 Summary of Performance Analysis

1. **Standalone Hashing:** The **Eijen** scalar implementation is already 3.6 times faster than the SHAKE-256 scalar baseline (for a 512-bit message); under 8-way parallelism, the AVX-512 implementation achieves a 15.9-fold speedup. The core advantage stems from a larger rate (1728 bits vs. 1088 bits) and fewer permutation rounds (16 vs. 24).
2. **Design Trade-offs:** The performance gains of **Eijen** arise from the Sponge-F mode, which allows for equivalent security at a smaller capacity, thereby freeing up more state bits for the rate. The combination of a large 2048-bit state and 16 rounds yields an effective absorption efficiency of 128 bits/round, which is significantly higher than Keccak’s 66.7 bits/round.

8 Randomness Tests

Since the five variants **Eijen- h** utilized the same permutation, we only evaluate the randomness about the full 2048-bit output of the permutation function Π . Three distinct types of inputs were tested:

- **Dense data:** The input messages consist of approximately 95% 1s and 5% 0s.

- **Sparse data:** The input messages consist of approximately 5% 1s and 95% 0s.
- **Random data:** The input messages are generated uniformly at random.

For each input category, 1000 input streams are sampled and each stream includes of 1,000,000-bit, which were subsequently subjected to randomness testing. The implementation of the randomness tests and the selection of testing parameters strictly adhere to industry standard specifications. The results are shown in Table 8.

Table 8: Randomness test results for the 2048-bit output of the permutation function Π (significance level $\alpha = 0.01$; pass threshold ≥ 981 out of 1000 samples)

Statistical Test	Dense data		Sparse data		Random data	
	Pass Rate	Result	Pass Rate	Result	Pass Rate	Result
Monobit Frequency	993/1000	Pass	988/1000	Pass	992/1000	Pass
Block Frequency m=10000	993/1000	Pass	989/1000	Pass	987/1000	Pass
Poker m=4	989/1000	Pass	996/1000	Pass	989/1000	Pass
Poker m=8	989/1000	Pass	993/1000	Pass	985/1000	Pass
Serial m=3, P1	993/1000	Pass	989/1000	Pass	993/1000	Pass
Serial m=3, P2	993/1000	Pass	986/1000	Pass	996/1000	Pass
Serial m=5, P1	991/1000	Pass	993/1000	Pass	994/1000	Pass
Serial m=5, P2	989/1000	Pass	995/1000	Pass	990/1000	Pass
Runs	990/1000	Pass	991/1000	Pass	993/1000	Pass
Runs Distribution	991/1000	Pass	988/1000	Pass	988/1000	Pass
Longest Run of Ones m=10000	992/1000	Pass	995/1000	Pass	989/1000	Pass
Longest Run of Zeros m=10000	989/1000	Pass	995/1000	Pass	986/1000	Pass
Binary Derivative k=3	987/1000	Pass	989/1000	Pass	990/1000	Pass
Binary Derivative k=7	992/1000	Pass	987/1000	Pass	992/1000	Pass
Autocorrelation d=1	990/1000	Pass	991/1000	Pass	993/1000	Pass
Autocorrelation d=2	993/1000	Pass	991/1000	Pass	994/1000	Pass
Autocorrelation d=8	989/1000	Pass	985/1000	Pass	989/1000	Pass
Autocorrelation d=16	991/1000	Pass	988/1000	Pass	990/1000	Pass
Binary Matrix Rank	992/1000	Pass	991/1000	Pass	990/1000	Pass
Cumulative Sums (Forward)	994/1000	Pass	988/1000	Pass	988/1000	Pass
Cumulative Sums (Backward)	992/1000	Pass	989/1000	Pass	991/1000	Pass
Approximate Entropy m=2	993/1000	Pass	989/1000	Pass	993/1000	Pass
Approximate Entropy m=5	996/1000	Pass	990/1000	Pass	991/1000	Pass
Linear Complexity m=500	990/1000	Pass	994/1000	Pass	992/1000	Pass
Linear Complexity m=1000	986/1000	Pass	995/1000	Pass	990/1000	Pass
Maurer's Universal L=7, Q=1280	986/1000	Pass	995/1000	Pass	988/1000	Pass
Discrete Fourier Transform	989/1000	Pass	987/1000	Pass	988/1000	Pass

References

- [1] Chun Guo, Kai Hu, Shuntian Jiang, Yanhong Fan, Yong Fu, Bart Preneel, and Meiqin Wang. Permutation-based hash from non-idealized assumptions: Adding feed-forward to sponge. Cryptology ePrint Archive, Paper 2025/1006, 2025.

- [2] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance. Cryptology ePrint Archive, Paper 2025/963, 2025.
- [3] Kai Fu, Meiqin Wang, Yinghua Guo, Siwei Sun, and Lei Hu. Milp-based automatic search algorithms for differential and linear trails for speck. In Thomas Peyrin, editor, *Fast Software Encryption - 23rd International Conference, FSE 2016, Bochum, Germany, March 20-23, 2016, Revised Selected Papers*, volume 9783 of *Lecture Notes in Computer Science*, pages 268–288. Springer, 2016.
- [4] Ling Sun, Wei Wang, and Meiqin Wang. More accurate differential properties of LED64 and midori64. *IACR Trans. Symmetric Cryptol.*, 2018(3):93–123, 2018.
- [5] Lars R. Knudsen and David A. Wagner. Integral cryptanalysis. In Joan Daemen and Vincent Rijmen, editors, *Fast Software Encryption, 9th International Workshop, FSE 2002, Leuven, Belgium, February 4-6, 2002, Revised Papers*, volume 2365 of *Lecture Notes in Computer Science*, pages 112–127. Springer, 2002.
- [6] Yosuke Todo. Structural evaluation by generalized integral property. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 287–314. Springer, 2015.
- [7] Yosuke Todo. Integral cryptanalysis on full MISTY1. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part I*, volume 9215 of *Lecture Notes in Computer Science*, pages 413–432. Springer, 2015.
- [8] Zejun Xiang, Wentao Zhang, Zhenzhen Bao, and Dongdai Lin. Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 648–678, 2016.
- [9] Ling Sun, Wei Wang, and Meiqin Wang. Automatic search of bit-based division property for ARX ciphers and word-based division property. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 128–157. Springer, 2017.

- [10] Kazumaro Aoki and Yu Sasaki. Preimage attacks on one-block md4, 63-step MD5 and more. In Roberto Maria Avanzi, Liam Keliher, and Francesco Sica, editors, *Selected Areas in Cryptography, 15th International Workshop, SAC 2008, Sackville, New Brunswick, Canada, August 14-15, Revised Selected Papers*, volume 5381 of *Lecture Notes in Computer Science*, pages 103–119. Springer, 2008.
- [11] Yu Sasaki and Kazumaro Aoki. Finding preimages in full MD5 faster than exhaustive search. In Antoine Joux, editor, *Advances in Cryptology - EUROCRYPT 2009, 28th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cologne, Germany, April 26-30, 2009. Proceedings*, volume 5479 of *Lecture Notes in Computer Science*, pages 134–152. Springer, 2009.
- [12] Xiaoyang Dong, Boxin Zhao, Lingyue Qin, Qingliang Hou, Shun Zhang, and Xiaoyun Wang. Generic mitm attack frameworks on sponge constructions. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IV*, volume 14923 of *Lecture Notes in Computer Science*, pages 3–37. Springer, 2024.

A Round Constants

Table 9: Round constants used in MixQuad

Round	$ci_0^{(r)}$	$ci_1^{(r)}$
0	0x243F6A8885A308D3	0x13198A2E03707344
1	0xA4093822299F31D0	0x082EFA98EC4E6C89
2	0x452821E638D01377	0xBE5466CF34E90C6C
3	0xC0AC29B7C97C50DD	0x3F84D5B5B5470917
4	0x9216D5D98979FB1B	0xD1310BA698DFB5AC
5	0x2FFD72DBD01ADFB7	0xB8E1AFED6A267E96
6	0xBA7C9045F12C7F99	0x24A19947B3916CF7
7	0x0801F2E2858EFC16	0x636920D871574E69
8	0xA458FEA3F4933D7E	0x0D95748F728EB658
9	0x718BCD5882154AEE	0x7B54A41DC25A59B5
10	0x9C30D5392AF26013	0xC5D1B023286085F0
11	0xCA417918B8DB38EF	0x8E79DCB0603A180E
12	0x6C9E0E8BB01E8A3E	0xD71577C1BD314B27
13	0x78AF2FDA55605C60	0xE65525F3AA55AB94
14	0x5748986263E81440	0x55CA396A2AAB10B6
15	0xB4CC5C341141E8CE	0xA15486AF7C72E993

B Test Vectors

B.1 Test Vectors for Eijen-256

- All-zero input:

```

0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000

```

The 256-bit digest:

```

80228A17A23CEAB 2887F1A68F207F64 CF37AFDE64829FB7 B51A409CE71378A7

```

- All-one input:

```

FFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF FFFFFFFFFFFFFFFFFF

```

The 256-bit digest:

```

7E3B2A5EE4CF56D3 FF4646C9A3577E97 059933A754462ECA CDE04218D1AE0004

```

- Input string “*abc*”, its ASCII representation is:

```

616263

```

The little-endian representation of the padded message can be expressed as:

```

0000000001636261 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 8000000000000000

```

Intermediate 2048-bit value of the iterative compression:

0	0000000001636261	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
	0000000000000000	0000000000000000	8000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
1	447370032E8A1913	C57DB45F9F4A43BB	22D5C22296D21D9E	251D6948B8A80863	447370032E8A1913	3AA99F3E3CA818C8	0AA913447370034E
	ECBFA165AB2807A3	1AAA6BA8FBEA30E4	F57DBC30D604233D	7A7A32AADA9E005C	D30827908EFC4424	1AAA6BB0FBCA30E4	D57E48BE116A0B11
	D308A385886A3F24	95DF6BFE71AA08E3	F9709B6E3D26698D	C57DB45F9F4A43BB	D308A39D88463F24	7A7A32AADA9E005C	65BC4A10D8440B91
	FF06704DA40A5DB8	7A7A32AADA9E005C	8A1913447370032E	1AAA6BB0FBC630E4	F4B172B5E7881993	7A7A32AABA9E005C	8A1913447370032E
2	5A6C0C8D85C45D06	DA02F53D1C3FDE40	DAE59F655DC5DAD6	27DFDE45E8568C13	9894C8A0A338F945	94095FE298E84C4A	641E02329F8A8C49
	E18ACB8FE7A6E2FF	BD760CF7DB9F07EF	B943BC10ACDA8566	E210789E43DDBB23	68E8ABE6F59A02F8	EB3664C315FEE8A0	A087F49DB9E3F1ED
	E82CC873BB7CA41A	F68A6BF1EBF11C01	C611798C8A470C26	E912FF3640816BDB	D6B577F33703B487	72A5603AE877A600	6B591A6FFBEC6C39
	0C2746FAC2B960A2	4463E486E9DE4F0C	05C8FDBCED952B87	84FBE313BDD55DB7	81B4BDA2F87791FF	4FE4830AA5F38ED3	5E63FFCFC9555054
3	1F41759E95376BEB	620BC03E60B943AB	4C8104571EB4AF90	0CC82291A7136721	AEDACCD335EB8BBD	66089E61581FE0C0	8EFE34D78E21D3AC
	2F520C0823F2AF04	85BBFE40728D1489	E3E2069CD61D3273	A2A03F3DEDB81B9E	939EAF36D4DF5EA	A90749CCBA9F7DAA	468FD6F06B68E968
	F3152682BAE3DAEA	CECA6BD28D0F2774	74F22330158850CF	64F6CB2AECE46634	222813B4230CE5BC	4B3676C42C668A1A	886EE679240610A7
	E152ABD16C9737E1	0936121385562D94	BAC2D07188D7ACC8	E6CC9D53B09B1FAA	D231B7C9ACBAD6F5	58013EF72A3CFAC1	EA427B210DBF4E15
4	D6FCCF3F0F5B1FF8	66DDAF18C8FF7F56	98AA6CF5496FF2A4	B58FEE234C538043	C806ED3445FD741D	E717A75CF3CD780A	1F4FBA2A1C2886D7
	A21D2805CE559CFA	E253A08CD4E22798	299D8DA21B9E1C96	E890BD45EF57B04C	635086AFD158D1B6	4959970BAEED3107	45C2EB7EE2C7FF9E
	BDC4683E0530DE0F	F40BFA7224DD130	046F6AB7E134FC24	10EEDCD66D1197A8	8D30571F9840AA82	D7A4F499902FA160	D25E4DF70513112E
	D6509A1DE2DCB62E	751C6342686A8D94	A8593A873A3A7EB2	2B44E202F84545A0	8BB662BA090F0674	697381B61834D371	20AF54301DE8E1A3
5	B5D80C518C405CD9	1F9807535085594F	154387D9F3E7E01E	B1C3145B16BF0A41	9DDFE17D89DB5960	83F5849A26B6C8BD	F80743EAE6FC9013
	368AC2DABB472739	084A5AB08140566A	B2390B86A1FC96C1	FFD137C0E10E37BF	0CD00E253789711B	F6DCEEF4846C0F5C	C704C17F0B2A1E94
	4AE6FCDC4521783F	0C4DAD059D85E3C5	9911EA8737E34FA8	1FF6499053DEA517	4B6C76F50F51D196	E3A43298A4D81BFB	6756182778FED707
	3B83C940BE28141C	E90001BB81C0E3DB	A9E18F5B9B9BF48F	782B819BC3FB2A15	9D9DE133B604C3C6	1A685B0716A7E68A	49D870B9D6CE86C4
6	9B79F361FB003CFC	50D1F9E7F5C3E0A3	2FCF99DF11C01688	B0FB1CF40EF4BFB1	46E32EAF3CF4DCE77	9ECE7C5AE512F321	9F152AFBEFBCF6D6
	DDb13A87B4A81179	104F8FCEB2488432	D358404361E392E7	C8C342EB6BB5C577	1FF0AD7574B9BB65	AAC662F1A658550B	8A47D49C742C3A75
	D93E33D9AD594FA0	693E3FF9FD9212F1	BABD18A3E8808800	F23A804B71353BE9	E7537AF7DE7E26B5	01DD1A88CD400E39	739089D23EE85639
	F26D13EC28FA8C42	EB9C613D2247813C	B6D255D6F2470E10	4B106AB482D430E9	5E6682442E21A6BE	0F7A0AA9397ECBD0	8B9690C5691160A7
7	3160823D2806CBF	B2734F094B955F12	CED93A87C3CCE2FF	E9A5A8373683BAED	2B02A1F93129F3C2	52EE67A014564CCE	1390AFA5777D9BB7
	66373E76C61B60F7	392C02BB1996D0B4	096D209780D83A40	2463245B6385D216	AFB0ACAC6C90C172	C0C54DED8DFFB254	AB13D3AEF29DE15B
	9FB6C1F939DFC559	5277D48665034542	319216A42A11A597	304EA9CD11E9F428	9A21F6EE8A3FD663	797D10BB5B179ED9	BD824DE3004B7978
	A1C7B8763DE85BBE	265C60F72B1566DE	6A40FE439781805E	6089F863D6C79CF4	A01BA95A8B9A660A	F84BD4A87FCF69DC	6C4042C578251404

8D16410AF6A1F8BD

8	DA1551D045BB6FF3 C8A90B6B1D850180 3D68F10D2F5C74A8 AF8253A4F8F6FF82	821954D7C27A34CF 7F42138C3255E4CF 559E19046EC4AF2F 05B7C022FBE5AEC5	AOA567954D14DFE2 FB9DE28B9FF0A428 68E5A806E53E0552 07349C1E5A4632EC	142F893A4B940892 05366E0097D577FB 186AFF2CE511A3BE 785C1EDE3F57CA1B	81A4C40099367699 198FB92E345819BA 3DC362C772E03BC5 2A2A60DF15FBE1DC	ED46B1A5596DDA83 5C523F8D7CA0FF28 67141809990A6055 B67435B05CBFEE6C	F27936B7631E0577 B3D896B896841F00 7FB60FF1F8A4549C ED943C9F884579CE	E8266E8A6AF23797 EF8EE136359615EC 9487646CADCDB09B E92BC96106A28FB3
9	AFFF5D22D0EA6333 29E6CCF07C3D5EB9 EO6416D577A2D937 1FD12615D1C1288C	39CDB470B2067AFB 7DA36E13B0A3C8B4 1EFDEF788EA9DCDF 5E5CA78493D762E4	63BD3E4911841A06 089AC5D10CD18A28 931EA6E97B3A9B4C 0FC020ADB0E8E3C8	74FEF34C8EC2AB57 301BF6F9D1789FA1 42968FF7F78C25AF FOA14F56B4DCEC9E	25E2A5EF3B049809 194AC54A4A8106C7 924585868C479F60 5295AA72C80A2913	EDC5ABBCB62E8E05 03D72E68B054DE94 BB4CBB3117686E71 D145B16ACC80F7DA	38253E84C1C978A9 845D7C698FE9F71F AA30B42B75CB5E4A 2BCF96B81ED4A87A	2FB9B655009FFB07 1CC2F156FA88EFD FCEF14579AA256A0 DDC30579D9F66E04
10	89B9CC299CC426C3 B141B7B60821F3E8 CD1E748CD05CA9CD 93B0D0BBF655300C	1AF1F213D163FDAD 52FA9BCC7B582C68 7CDD41A4ABB6F778 EAD7E2E2DDE23011	2859CF0E26ED6EE9 7DA9878EAF3D19A9 A56DED5A86CF9140 CBEE4F4A1F76D545	42E5647951720E73 A562B329BF8BDBF6 F5C07EB2C931FC3E 45EAF5650D079D8	71E99A047ABE0A90 0EC7D52DE60400E9 CC6E608DA6C0B92F CDB4B27A28CD353B	E23AF5F3CB1152CC 71427E995EA56E3E 881B3A03D78272F9 857C6ABB7E7D0BAE	95BDB2BA117ADE05 A955BC56311B2B2B D64A149101EB071B 0803444AC1D4183D	DFBC2C0982CEE402 E77E044EEAABEE84 09070894015B291D 595EFE7233D880F8
11	A59D5327A8069F12 9E12F5691CF115BC CA5B615E988BDBE1 4C849FD61DB9A26E	8C503E479F1D1892 686BCFC4DFA14338 5135165F60A2BF25 57D9B36DBC04076A	CD61C7E30C7F17C5 78937C5BF4B999E1 86E23AA5F168FC84 4D9910414890C2C4	2193424BC2F3BA64 3AB195793B1D429B CF5FA4598E92124D FCB06E37E2572B06	1A92D8122D38E24F 4549D2A478542064 2F6CD1006C531A6D 74F8FF24AEC69296	D21477116881B6A5 3807C783B2FC116D C269E401C5F6EA7B 5985ED0E1C2199E7	AC534D6815B222B8 EA8ABE943D6A38D0 38FE46BF1D3A950F AA3A8C963F0365D7	7DE4C2F124E58DE6 B5BEDB7A536A286C DCCEC2CD0EBF8F5E 416DOCDDA1AD0EE
12	93D03F22EE36E665 20A7F50D162B37D5 74504DD1259BB12F D4A26C7026F8D356	8E18D01F31D9E7B8 293DC2C9FA7DFD5C DED165072AE1693D CA3ABA9BE312D5FA	C5D7C24F06B02231 4ED7CCB6EC7871D2 FABF8874E5DE1787 74BCB0DFB6194E18	695F01CE2BA0B18A D92612B5CD025811 E63532D53CC66EC4 EA6A92BAA967FA33	5C7EA03DFA41CA8F BDBDE4A83FC6C17C 813FB32A1F650ACC 2EFB72F44AD7A5F5	345ED968AD4B12C3 125011C6B4FA7278 2CDDCAA1F2272BAA0 D51983180FC34B2E	9A59264AC992C351 7FDCFB2C91FE7978 ABE61012CFDDDCB5 9D118B97DBA99907	5F433E61045E4AA2 43ADACD0EFC9DAC CF2D9F0EB4E8A05F B117582EBA3B75CA
13	12454AB21A1DD9EE E9A1D42BB6FDD9DB 0A47886A6D96CBC2 AB1B66EF45DC4EBA	AF1649249B7F7379 2DAAA04DCFB375E5 D36BF33199B319A8 E2759E96CB7C5A4B	CFD2B607AE22CCAB 09570C95DCB6B88E 4648ADAF1E9A8164 D69E349F579AD45B	4FB45ACEB46FA699 913698392571E01E 31151CA3E47A0CEE 77CC3306A367157B	D58FB51E7E7987B7 E80F680358C4CA9A 0C3ED06DE7882552 A443A6222FA82028	6F588226CCA8F8D1 7A7A6700D729C843 9C818A2E92F335C0 809F184486BCEE50	C0990A4943F652A8 7F1574DFE1AC35B3 OFF0E0C71DC2DE41 26D89AA676196DC0	CEB7A570D37DFE62 1FCE483F93F46024 9A8CA497FCDDB2C1 B1A7A01080BEDF00
14	2FBE163F5A01B065 11C6CB0A035CA660 103F35072294B898 4551C031ACF52B6E	F1802381D1021B77 021330607B11913E 8349A6ED1C79B19A D57A956977674337	A2ACAEDB09810F6E 5DC16200DD0EEF4A AF1FBB04E9AF3D86 E5C0FE8958DC2A44	AF6150F073BC3F82 7086B867913D7DD3 4340FD08B68971C7 3D71059004127F63	0D04F97EFF07F333 A54938B2D73022AE AFA1877AF50037DB E98C02151941FFBF	96F43D7D584AB324 6F584452159A4405 49589F4231EE4A1C 06E7B57EC0958AE7	A47BA1625DCFAB01 9E274753767DED18 C321CF305FEA6E12 D19012C289F182DD	8E77A950239B6BD8 E62852BD85585459 94DCF4F96D85B17C F3EB0AA1E2564EE5
15	49B957C36CE8E224 FE70DAEEF5F26128 D15A70D06F8AA2C2 FO33B3D6CB4873F0	44565A3380D64C46 0B4E0A4662742048 9166C9E603754A4E C28D4CFADC58DA8D	7151588634DC93FC A50B8EB5F99D2EB4 1DBDBD54D8D1753D 947D321FDA48B83E	8239FD3CDE73D6EE A70B2680E149B3DE CF86B839A5354A33 FAA8365E908EDFED	96F71857CCABDDC4 538D1C600067EE89 7C50733A605A8613 F9BCFA40CBD5A620	EF2AE43D50CFC2E8 6200606D39810216 D3B6EDA703AE49C8 EE26A078495DFF8C	A657C27732823687 3F80FBD3A8320A08 DECAA90B81DFE7D5 89A68027595F081E	C6B31046656F43BB E4DB721781C3BCA8 EC773E71A46B3396 72D5F023CCE7D9E8
16	09F929059CB0A856	57F44A45657E82D4	B12D6FB71368CC7E	3D141A671C16D22E	35AA302A8641EE6C	OBADD548E71CBCE6	204B732FF45E25F1	28D581191EED2DAE

3D1EBE978D0F89C6	8BFFC212115EB6A9	2A9D9DEF0209689F	51E3488BB0CF95E5	7FAE02FDD22920E1	AA1F827DA56D0D0D	64F73F78B7BA6A6A	E558857C9139EB0F
738521FE5D9A3EB4	A08418A650071F29	D832089843253040	64AAE1A2402E5B30	FC9E471C3278C941	6638EE7D845FF6F1	CE5EF11EABAA83B	444CB95EAC455791
B0F1FC89DFF89974	05CEAE73C34D9A97	A03123E2B9D22A23	02B49E012D880194	C832C8FBDAB9CBB6	0824D2E879ACE7CE	B1F902E54FCAA9DC	81E09F3C71B04F86

The 256-bit digest:

C832C8FBDAB9CBB6	0824D2E879ACE7CE	B1F902E54FCAA9DC	81E09F3C71B04F86
------------------	------------------	------------------	------------------

B.2 Test Vectors for Eijen-384

- All-zero input:

0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000							

The 384-bit digest:

B073FF1383DED03B	501800B493D0A9D9	80228A17A23CBEAB	2887F1A68F207F64	CF37AFDE64829FB7	B51A409CE71378A7
------------------	------------------	------------------	------------------	------------------	------------------

- All-one input:

FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF							

The 384-bit digest:

AAB7CF11B814EB94	3A3702838F26095D	A09ADCDBBEE52337	24CCEE1FA4DC4B4D	A6A3EA682BF08970	BF19275E9FABBEFD
------------------	------------------	------------------	------------------	------------------	------------------

- Input string “*abc*”, its ASCII representation is:

616263

The 384-bit digest:

A03123E2B9D22A23	02B49E012D880194	C832C8FBDAB9CBB6	0824D2E879ACE7CE	B1F902E54FCAA9DC	81E09F3C71B04F86
------------------	------------------	------------------	------------------	------------------	------------------

B.3 Test Vectors for Eijen-512

- All-zero input:

0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000

The 512-bit digest:

20720A80BCA29353	9A5310A46EFA2926	B073FF1383DED03B	501800B493D0A9D9	80228A17A23CBEAB	2887F1A68F207F64	CF37AFDE64829FB7	B51A409CE71378A7
------------------	------------------	------------------	------------------	------------------	------------------	------------------	------------------

- All-one input:

FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF

The 512-bit digest:

3B175FF348C41499	955B22A6E54982D8	08847F0E2FE7C269	B51E1DCA1C1FF9C8	25897C9F871AFFE4	11E0B646553C2F16	C2E5DD95F186CA01	3D4F2019FBAA0174
------------------	------------------	------------------	------------------	------------------	------------------	------------------	------------------

- Input string “*abc*”, its ASCII representation is:

616263

The 512-bit digest:

B0F1FC89DFF89974	05CEAE73C34D9A97	A03123E2B9D22A23	02B49E012D880194	C832C8FBDAB9CBB6	0824D2E879ACE7CE	B1F902E54FCAA9DC	81E09F3C71B04F86
------------------	------------------	------------------	------------------	------------------	------------------	------------------	------------------

B.4 Test Vectors for Eijen-1024

- All-zero input:

0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000
0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000	0000000000000000

The 1024-bit digest:

B3F4CB5B73AB79B2	0EDC62546C428718	A1049A487EF76599	FC96E7B6494278A9	C3DD02B9FE08D75E	316A87479AE6A8F4	7CB4F076F3ED6B07	849B986B2CF1689B
20720A80BCA29353	9A5310A46EFA2926	B073FF1383DED03B	501800B493D0A9D9	80228A17A23CBEAB	2887F1A68F207F64	CF37AFDE64829FB7	B51A409CE71378A7

- All-one input:

FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF

The 1024-bit digest:

10CA8AA88377EB4A	7487213EE0384BBA	5CBB5A9300C000CE	9192DE12D5D81E85	535A50AAD23556E4	F795F11F73702C7B	FF42DB838BE183B1	B1A31D9C96B025BD
47B0CD040C6B45CC	BC54F8D7B3BEB6CE	4C4D2BBDACB1D7EF	7918BD2E0912F86E	895BFDE5EB24DCFA	1EE5124527E17473	B73B1C6FFE41F781	0CF8C022BA818EEA

- Input string “*abc*”, its ASCII representation is:

616263

The 1024-bit digest:

738521FE5D9A3EB4	A08418A650071F29	D832089843253040	64AAE1A2402E5B30	FC9E471C3278C941	6638EE7D845FF6F1	CE5EF11EABAA83B	444CB95EAC455791
B0F1FC89DFF89974	05CEAE73C34D9A97	A03123E2B9D22A23	02B49E012D880194	C832C8FBDAB9CBB6	0824D2E879ACE7CE	B1F902E54FCAA9DC	81E09F3C71B04F86